

Amazon Cloud Storage Services

We explored Amazon EC2 and compute services in the previous chapter and provisioned EC2 instances in the Amazon cloud, including Windows and Linux instances. In this chapter, we will discuss Amazon cloud storage, including the block cloud storage that can be attached to an EC2 instance, the network filesystem cloud storage that can be shared by many EC2 instances, and the object cloud storage storing objects in the cloud. We will cover the following topics in this chapter:

- **Amazon Elastic Block Store (EBS):** Provides block-level storage volumes to EC2 instances. We will show how to create and attach storage volumes to EC2 instances and use them as primary storage.
- **Amazon Elastic File System (EFS):** Provides scalable and fully managed filesystem storage to be shared by EC2 instances and on-premises resources.
- **Amazon Simple Storage Service (S3):** Provides object storage that can store and retrieve any amount of data from anywhere on the web.
- **Amazon Snowball and Snowmobile:** Physical data transfer services for transferring large amounts of data into or out of the AWS cloud.
- **Accessing S3 from EC2 instances:** By leveraging an EC2 IAM role, EC2 instances can easily access S3 and take advantage of S3's scalable, durable, and highly available storage services for your applications running on EC2.

Following the discussions in this chapter and integrating the EC2 knowledge and skills learned from the last chapter, you will be able to dive deep and understand why we need to and how to create block storage, network filesystem storage, and simple storage in the cloud, how you can transfer storage from your on-premises data centers to the cloud, and how to access the cloud storage from an EC2 instance.

Understanding EBS

When we run out of storage on our home PC, we usually buy a new disk drive, shut down the computer, install the disk drive, reboot the computer to recognize the new disk drive, and then log into the computer guest OS to format the new disk drive and start using it.

This type of disk storage is called *block storage*, which is a technology that controls storage on the computer, using a block as the storing unit. It takes the data to be stored, divides it into blocks of equal sizes, and stores the data blocks on the underlying physical storage. With block storage, a block is the unit for data storing and retrieving, and only changes to the blocks are written to the disk. For example, when you change a sentence in a Microsoft Word doc and save the doc, only the blocks that store the sentence were updated to the physical disk drive, in contrast to *object storage*, which uses an object (such as a fingerprint file) as a storing unit - any *partial* changes to the object will update the *whole* object. Object storage stores the whole object as a unit, which has metadata for object retrieval.

Block storage is usually formatted by the computer's guest OS to form a filesystem. If the disk drive is locally attached to the computer and formatted, it will be a local filesystem. A local filesystem can be shared with another computer through a network, and it is called a *network filesystem* from the remote computer's point of view. When you write to a file on a network filesystem, the changed portion is updated to the source disk drive via the network. The filesystem has a directory/folder hierarchy and stores files with blocks. Block storage uses disk drive blocks directly for reading and writing. The concepts of object storage, filesystem, and block storage are illustrated in *Figure 2.1*:

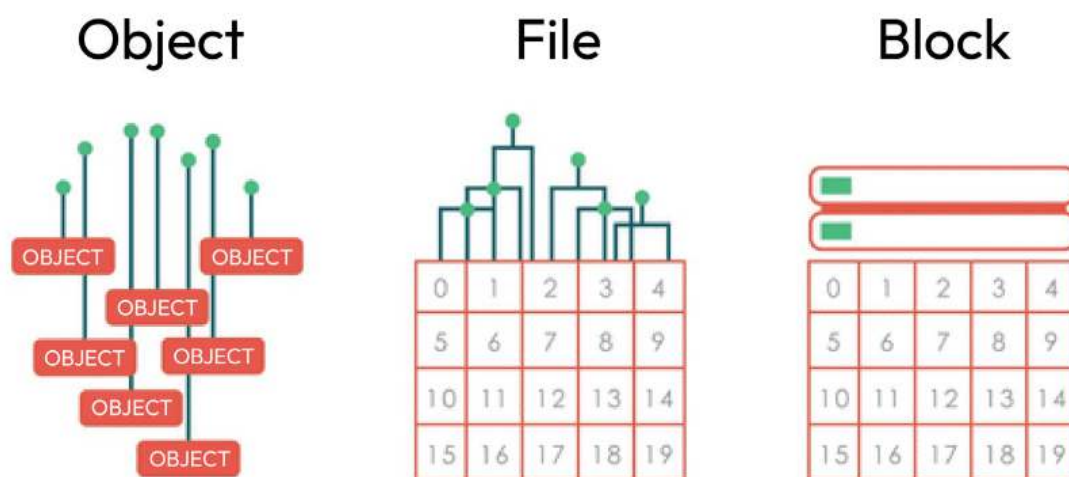


Figure 2.1 – Object storage, filesystem, and block storage

How do we add disk storage to an EC2 instance in Amazon cloud? Instead of buying a new disk drive for our home computer, we create a virtual disk drive in the AWS cloud, called an **EBS** volume. Instead of shutting down a computer and physically installing a disk drive to it, we attach the EBS volume to an EC2 instance on the fly (without shutting down the instance), and then format the disk from the guest OS. Much simpler than our earlier example of adding a physical disk to a PC, we create a virtual disk drive in the AWS cloud, called an **EBS** volume, and attach it to an EC2 instance on the fly.

With only a couple of clicks, we add disk storage to the instance and format it to use. Here are the detailed steps:

1. Log into the Linux instance and check its existing disk storage:

Log into the Linux instance we launched earlier, and run the following OS commands:

`sudo su` (become the superuser in the Linux system)

`fdisk -l` (list the disks visible in the Linux system)

As shown in *Figure 2.2*, the preceding command lists all the disks and one 8 GB disk drive, which is the root disk for the guest OS:

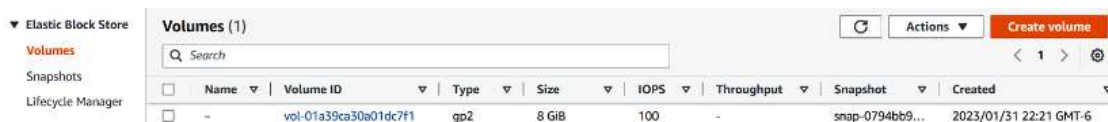
```
[ec2-user@ip-172-31-22-204 ~]$ sudo su -
[root@ip-172-31-22-204 ~]# fdisk -l
Disk /dev/xvda: 8 GiB, 8589934592 bytes, 16777216 sectors
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disklabel type: gpt
Disk identifier: 61856571-99DD-4505-96FC-1DD8820522F6

Device        Start      End  Sectors  Size Type
/dev/xvda1    4096 16777182 16773087   8G Linux filesystem
/dev/xvda128  2048    4095     2048    1M BIOS boot
```

Figure 2.2 – Check the Linux instance disk storage

2. Create an Amazon EBS volume:

Go to **AWS console | EC2 instance dashboard | Elastic Block Store | Volumes**, and we can see the root EBS volume of the instance, as shown in *Figure 2.3*. Click on **Create Volume**:



The screenshot shows the AWS Elastic Block Store Volumes page. On the left, there's a sidebar with 'Elastic Block Store' and sub-items 'Volumes', 'Snapshots', and 'Lifecycle Manager'. The main area is titled 'Volumes (1)' and contains a table with one volume. The table has columns for Name, Volume ID, Type, Size, IOPS, Throughput, Snapshot, and Created. The volume listed is 'vol-01a39ca30a01dc7f1' of type 'gp2', size '8 GiB', and IOPS '100'. It is associated with snapshot 'snap-0794bb9...' and was created on '2023/01/31 22:21 GMT-6'. There are search and pagination controls at the top of the table.

Name	Volume ID	Type	Size	IOPS	Throughput	Snapshot	Created
-	vol-01a39ca30a01dc7f1	gp2	8 GiB	100	-	snap-0794bb9...	2023/01/31 22:21 GMT-6

Figure 2.3 – The EBS volume for the EC2 instance

In the new window shown in *Figure 2.4*, fill in the details such as volume size, type, and AZ, then click **Create volume**. Based on performance and price, Amazon EBS provides three different volume types: **Solid State Drive (SSD)** volumes, **Hard Disk Drive (HDD)** volumes, and previous-generation volumes. More details are available from <https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/ebs-volume-types.html>.

EC2 > Volumes > Create volume

Create volume [Info](#)

Create an Amazon EBS volume to attach to any EC2 instance in the same Availability Zone.

Volume settings

Volume type [Info](#)

General Purpose SSD (gp2) ▼

Size (GiB) [Info](#)

10 ▼

Min: 1 GiB, Max: 16384 GiB. The value must be an integer.

IOPS [Info](#)

100 / 3000

Baseline of 3 IOPS per GiB with a minimum of 100 IOPS, burstable to 3000 IOPS.

Throughput (MiB/s) [Info](#)

Not applicable

Availability Zone [Info](#)

us-west-2a ▼

Snapshot ID - optional [Info](#)

Don't create volume from a snapshot ▼ 

Encryption [Info](#)

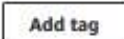
Use Amazon EBS encryption as an encryption solution for your EBS resources associated with your EC2 instances.

☐ Encrypt this volume

Tags - optional [Info](#)

A tag is a label that you assign to an AWS resource. Each tag consists of a key and an optional value. You can use tags to search and filter your resources or track your AWS costs.

No tags associated with the resource.



You can add 50 more tags.

Cancel **Create volume**

Figure 2.4 – Create an EBS volume

3. Attach the new EBS volume to the Linux EC2 instance:

Now, we will see two EBS volumes. Choose the newly created volume and select **Attach volume** from the drop-down **Actions** menu, as shown in *Figure 2.5*:

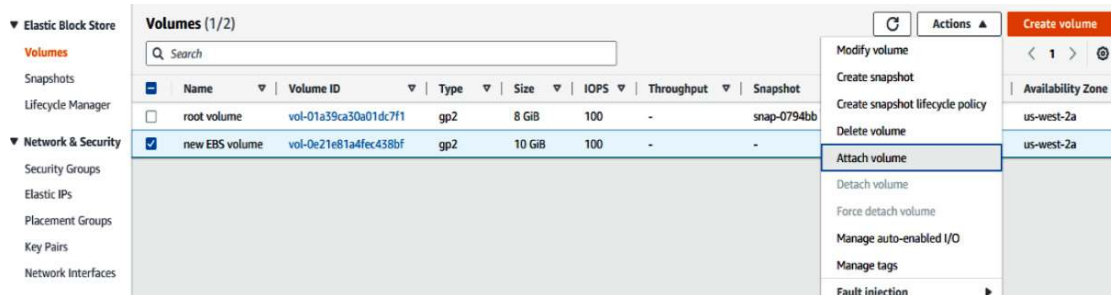


Figure 2.5 – Attach the EBS volume

Select the EC2 instance we created earlier in the pop-up window. Click **Attach Volume**, as shown in *Figure 2.6*:

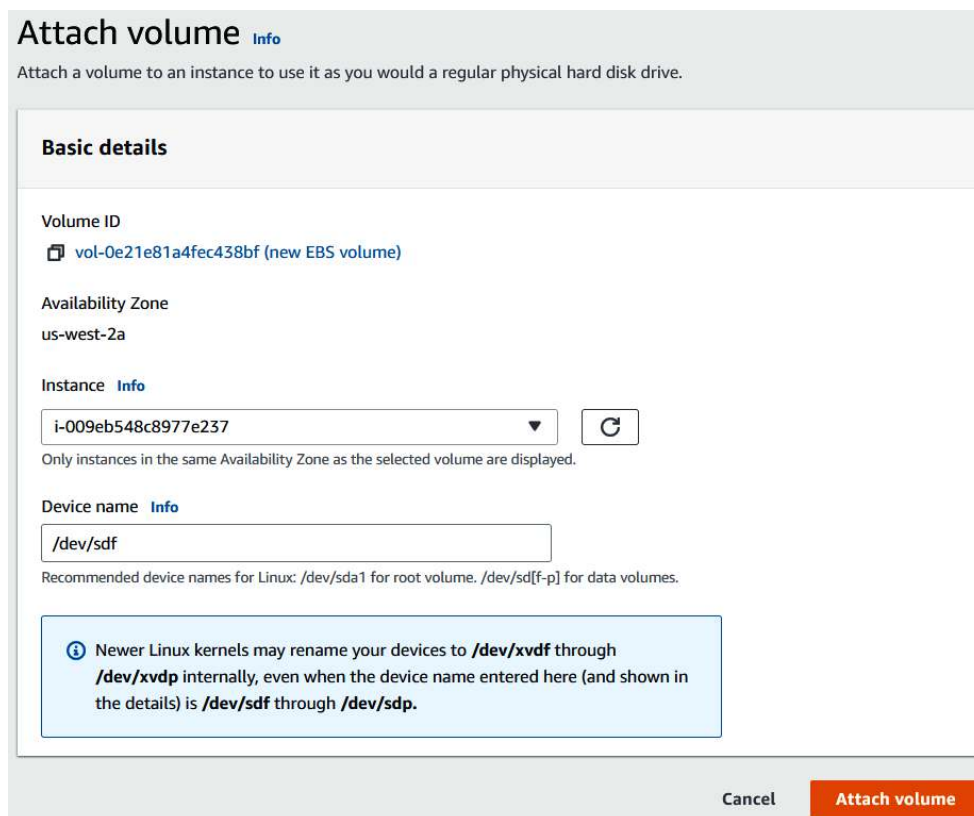


Figure 2.6 – Attach the EBS volume to the EC2 instance

4. Go back to the Linux Guest OS, format the new disk, and use the storage:

Afterward, the EBS volume is attached to the instance. On the Linux Guest OS, run `fdisk -l` again. We now see two disk drives, including the new 10 GB disk drive. Run the Linux command `mkfs` to create a new filesystem from the newly added disk, make a new directory `/ebs1` and use `mount` to mount the filesystem to `/ebs1`, as shown in *Figure 2.7*. We are now using the new disk storage:

```
[root@ip-172-31-22-204 ~]# fdisk -l
Disk /dev/xvda: 8 GiB, 8589934592 bytes, 16777216 sectors
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
Disklabel type: gpt
Disk identifier: 61856571-99DD-4505-96FC-1DD8820522F6

Device            Start      End  Sectors  Size Type
/dev/xvda1         4096 16777182 16773087    8G Linux filesystem
/dev/xvda128       2048     4095     2048    1M BIOS boot

Partition table entries are not in disk order.

Disk /dev/xvdf: 10 GiB, 10737418240 bytes, 20971520 sectors
Units: sectors of 1 * 512 = 512 bytes
Sector size (logical/physical): 512 bytes / 512 bytes
I/O size (minimum/optimal): 512 bytes / 512 bytes
[root@ip-172-31-22-204 ~]# mkfs /dev/xvdf
mke2fs 1.42.9 (28-Dec-2013)
Filesystem label=
OS type: Linux
Block size=4096 (log=2)
Fragment size=4096 (log=2)
Stride=0 blocks, Stripe width=0 blocks
655360 inodes, 2621440 blocks
131072 blocks (5.00%) reserved for the super user
First data block=0
Maximum filesystem blocks=2684354560
80 block groups
32768 blocks per group, 32768 fragments per group
8192 inodes per group
Superblock backups stored on blocks:
    32768, 98304, 163840, 229376, 294912, 819200, 884736, 1605632

Allocating group tables: done
Writing inode tables: done
Writing superblocks and filesystem accounting information: done

[root@ip-172-31-22-204 ~]# mkdir /ebs1
[root@ip-172-31-22-204 ~]# mount /dev/xvdf /ebs1
[root@ip-172-31-22-204 ~]# cd /ebs1; touch x
[root@ip-172-31-22-204 ebs1]# ls
lost+found  x
```

Figure 2.7 – List disks and format the new disk

5. Increase the EBS storage as needed

After an EBS volume is attached to the Linux instance and made a useable filesystem by the guest OS, we can increase the EBS volume size and extend the filesystem at any time, with two steps:

- Increase the size of the EBS volume in the AWS console
- Extend the filesystem on the Linux Guest OS

More details are available at <https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/modify-ebs-volume-on-instance.html>.

In the preceding example, we created an Amazon EBS volume and attached it to the Linux instance, then formatted the disk to a filesystem, and used it from the instance guest OS. The filesystem is expandable at any time as needed. For EC2 instances with Windows guest OS, you will use the `format` command to format the new disk and label it with a drive letter such as D, E, and so on. More details are available from <https://www.bu.edu/comtech/students/technical-guides/hardware/how-to-format-hard-drives/>.

In the next section, we will discuss Amazon EFS and demonstrate how to create and use it for multiple EC2 instances as a network filesystem.

Understanding EFS

An Amazon EBS volume can be created and attached to an EC2 instance, but usually, it is not meant for many instances to share. Amazon EFS provides simple, scalable, elastic file system storage that can be shared among many EC2 instances on the AWS Cloud and servers on-premises. With only a couple of clicks, we can create and share Amazon EFS to EC2 instances easily. Here are the detailed steps:

1. Create an Amazon EFS filesystem

Log into the AWS console, go to the EFS service, and create a filesystem, as shown in *Figure 2.8*:

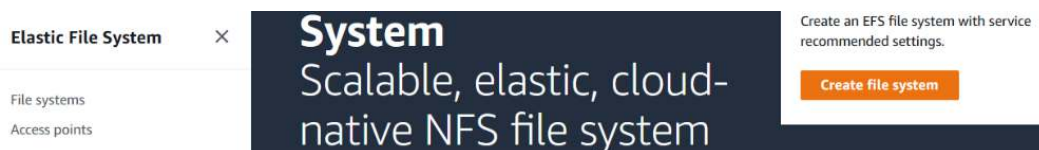


Figure 2.8 – Create the Amazon EFS filesystem (1)

I. Click "Create file system":

In the pop-up window, provide a name for the EFS filesystem and choose a storage class: a One Zone EFS filesystem stores data at the AZ level, and a standard filesystem stores data redundantly across multiple AZs in the same region. Note that there is no need to specify the size of the EFS filesystem since it scales elastically depending on the EC2 instances' demands on the shared filesystem. Click **Create**, as shown in *Figure 2.9*. The EFS filesystem is now created:

Create file system ✕

Create an EFS file system with service recommended settings. [Learn more](#)

Name - optional
Name your file system.

efs1

Name can include letters, numbers, and +-=._:/ symbols, up to 256 characters.

Virtual Private Cloud (VPC)
Choose the VPC where you want EC2 instances to connect to your file system. [Learn more](#)

vpc-0f123e6a
default

Storage class [Learn more](#)

☒ **Standard**
Stores data redundantly across multiple AZs

☐ **One Zone**
Stores data redundantly within a single AZ

Cancel **Customize** **Create**

Figure 2.9 – Create the Amazon EFS filesystem (2)

2. Attach the created EFS filesystem to a Linux instance:

Review the EFS filesystem and click the **Attach** tab, as shown in *Figure 2.10*:

Amazon EFS

efs1 (fs-0bdb2cdd4c14a8376) Delete Attach

General Edit

Performance mode General Purpose Throughput mode Bursting Lifecycle management Transition into IA: 30 day(s) since last access Transition out of IA: On first access Availability zone Standard	Automatic backups ✔ Enabled Encrypted 9835af2e-3c7d-4471-8ba7-980b02f4b317 (aws/elasticfilesystem) File system state ✔ Available DNS name fs-0bdb2cdd4c14a8376.efs.us-west-2.amazonaws.com
---	--

Figure 2.10 – Review the Amazon EFS filesystem

Then click the **Using the NFS client tab**, then copy the Linux `sudo mount` command, as shown in *Figure 2.11*:

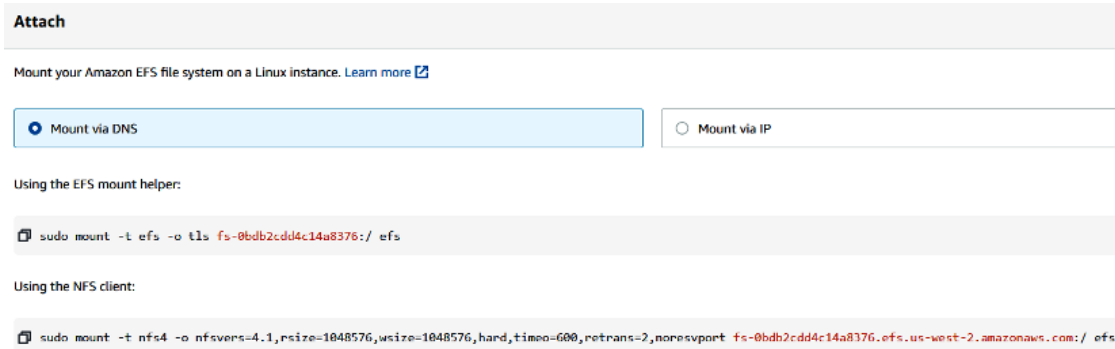


Figure 2.11 – Copy the mount command

Go to the Linux guest OS, use `mkdir` to create a new directory, and mount the EFS filesystem to the directory using the `sudo mount` command copied earlier. As you can see from *Figure 2.12a*, the new filesystem is now ready for reading and writing:

```
[root@ip-172-31-26-122 /]# mkdir /efs1
[root@ip-172-31-26-122 /]# sudo mount -t nfs4 -o nfsvers=4.1,rsz=1048576,wsz=1048576,hard,timeo=600,
retrans=2,noresvport fs-0bdb2cdd4c14a8376.efs.us-west-2.amazonaws.com:/ /efs1
[root@ip-172-31-26-122 /]# cd /efs1; cp /etc/hosts .
[root@ip-172-31-26-122 efs1]# ls
hosts
[root@ip-172-31-26-122 efs1]# touch x
[root@ip-172-31-26-122 efs1]# ls
hosts x
```

Figure 2.12a – Mount the EFS file system to a Linux Instance

3. Verify the EFS filesystem is shared from another Linux instance:

Log into another Linux instance in the same region, use `mkdir` to create a directory, and **mount the same EFS filesystem** to the local directory. You will find that the filesystem is shared among the two EC2 instances with read and write, as shown in *Figure 2.12*:

```
[root@ip-172-31-31-184 ~]# sudo mount -t nfs4 -o nfsvers=4.1,rsz=1048576,wsz=
=1048576,hard,timeo=600,retrans=2,noresvport fs-0bdb2cdd4c14a8376.efs.us-west-2.
amazonaws.com:/ /efs2
[root@ip-172-31-31-184 ~]# ls /efs2
hosts x
[root@ip-172-31-31-184 ~]#
```

Figure 2.12b – Verify the shared EFS filesystem on another Linux instance

With a few clicks, we have created an EFS filesystem and shared it among two EC2 Linux instances in the same region. In the next section, we will investigate the AWS object storage: S3.

Understanding S3

As we have discussed, Amazon EBS is block storage that can be attached to an EC2 instance. Amazon EFS is an elastic filesystem storage that can be shared among EC2 instances. Now, we will examine Amazon's object storage: S3.

S3 is object-based storage and is a public end point accessible globally via the web and other means. In S3, objects or files are stored in a bucket (*folder*). S3 is a universal namespace storage, which means the names must be unique globally. While there is unlimited storage for S3 customers, each object or file is limited to 0 TB to 5 TB in size.

Amazon S3 offers a range of object-level storage classes that are designed for different use cases:

- S3 Standard, with 4x9 (99.99%) availability and durability.
- S3 Standard-Infrequent (Standard-IA), with 3x9 (99.9%) availability and 11x9 durability.
- S3 Reduced Redundancy Storage with 3x9 availability and 4x9 durability.
- S3 Intelligent-Tiering, which places the objects to the right tier based on access activities.
- Glacier is for archive only, with 11 x 9 data durability. You can choose from different archive storage classes optimized for different access patterns and storage durations.

Amazon S3 offers versioning to manage different versions of an object in the same bucket, so we can easily recover an object from both unintended user actions and application failures.

Amazon S3 offers life cycle management that manages the automatic movement of objects between different S3 storage classes. With life cycle management, you can configure a set of rules that automatically transition objects from one storage class to another storage class at a specified condition, to minimize storage costs or optimize performance as needed. S3 life cycle management offers an easy way to manage object storage rate and cost for long-term storage and to configure actions and configurations for expiration, including the permanent deletion of objects.

Figure 2.13 shows an example of life cycle management: the current version of S3 objects will sit in Standard S3 class for 30 days, then transition to Standard-IA for 100 days, then move to AWS Glacier, and will be permanently deleted after 455 days:



Figure 2.13 – S3 lifecycle management example

AWS S3 storage provides a simple and easy-to-use storage service that integrates well with the other AWS services. But how to transfer a big amount of data from on-prem to the AWS cloud? We will answer this question in the next section.

Understanding Snowball and Snowmobile

To help customers transfer a huge amount of data (close to petabytes) to the cloud, Amazon offers data transportation services including AWS Snowcone, AWS Snowball, and AWS Snowmobile.

AWS Snowcone is a small, rugged, but secure and versatile device that can help organizations to extend their AWS infrastructure to the edge and enable them to do fast data transfers. With Snowcone, you can collect, process, and transfer data to AWS, either offline by shipping the device or online with AWS DataSync.

AWS Snowball is an appliance-based data transfer service that uses secure, portable storage devices that are delivered to your doorstep, for you to load your data and then send it back to AWS after the data transfer is complete. The data is encrypted during the transferring process, leveraging Amazon S3 for secure storage and Amazon **Key Management Service (KMS)** for encrypting the data on the device. An AWS Snowball appliance provides up to 80 TB of usable HDD storage.

AWS Snowmobile is a truck-based data transfer service that uses a secure truck with 100 **Petabytes (PB)** storage capacity, to be dispatched to your site, and connected to your network to perform high-speed data transfer. The truck will be driven back to AWS to transfer the data to your cloud accounts. During the whole process, data is encrypted at rest and in transit. AWS Snowmobile also provides users with analytics and reporting capabilities during the data transfer process.

Accessing S3 from EC2 instances

Now that we have launched an EC2 instance in *Chapter 1* and created S3 buckets in *Chapter 2*, naturally, we will ask the question: do my EC2 instances have access to my S3 buckets?

To answer this question, we need to look at it from two perspectives:

- S3 is a public endpoint, so the EC2 instance needs to have a public IP address.

However, that's not enough – when you log into the EC2 instance, and run the command (as shown in the following figure):

```
aws s3 ls
```

You will find that there are no S3 buckets listed:

```
[ec2-user@ip-172-31-28-114 ~]$ aws s3 ls
Unable to locate credentials. You can configure credentials by running "aws configure".
```

Figure 2.14 – No S3 bucket is found from EC2

- To have an EC2 instance access S3 buckets, we need to assign an IAM role to the EC2 instance. Recall that we briefly covered this in the EC2 section in *Chapter 1*:

An IAM role can be assigned with permissions to access other AWS resources, such as reading an Amazon Simple Storage Service (S3) bucket. By attaching the IAM role to an EC2 instance, all the applications running on the EC2 instance will have the permissions of that role.

Here, the applications running on the EC2 instance include a terminal/shell running on the EC2 instance. Let us implement this step by step, starting from an EC2 role creation:

1. Log into the AWS console, go to the IAM services, and click **Create role** as shown in *Figure 2.15*:

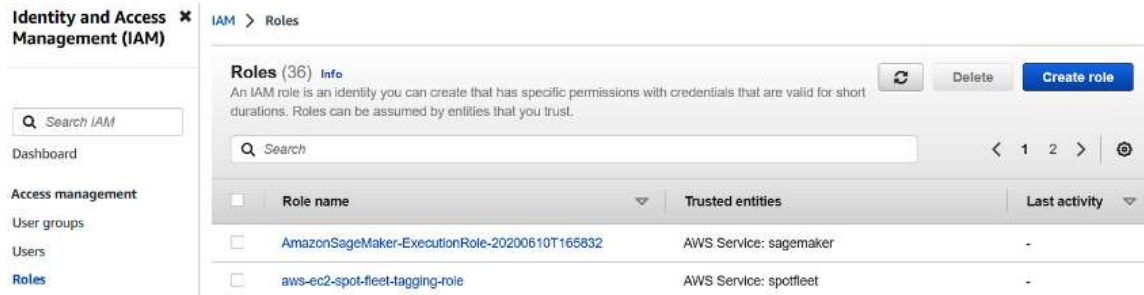


Figure 2.15 – Create an IAM role

- In the pop-up window, choose **AWS service** under **Trusted entity type**, and **EC2** as the use case, then click **Next**, as shown in the following figure:

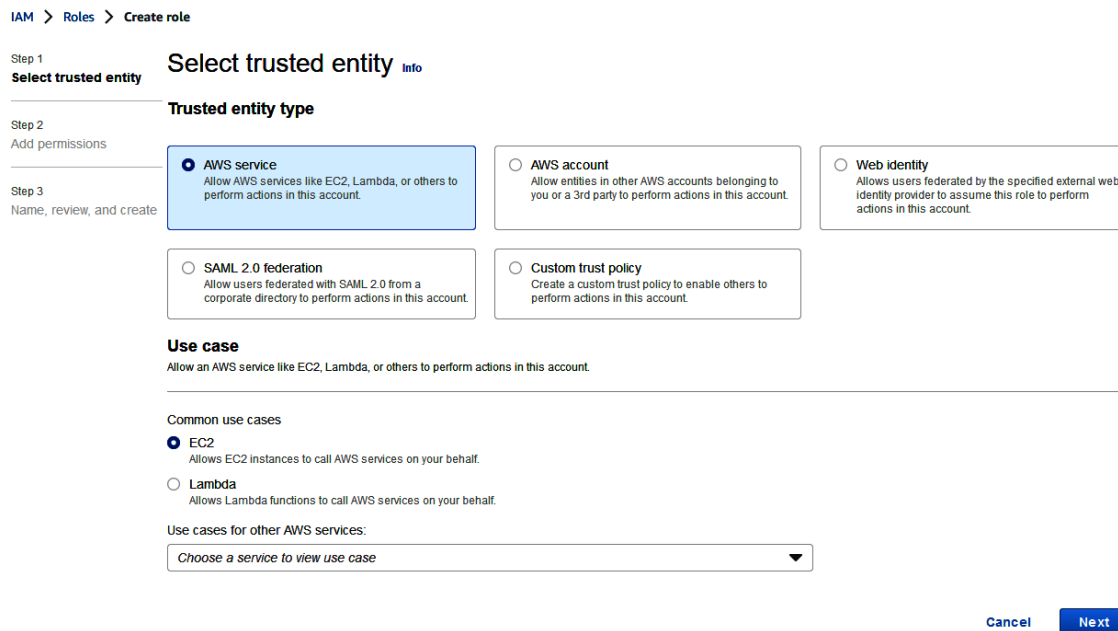


Figure 2.16 – Selecting a trusted entity type

3. In the **Add permissions** tab, type **AmazonS3** as the filter and then choose **AmazonS3FullAccess**, and click **Next**, as shown in *Figure 2.17*:

Step 1
Select trusted entity

Step 2
Add permissions

Step 3
Name, review, and create

Add permissions [info](#)

Permissions policies (Selected 7/814) [info](#)
Choose one or more policies to attach to your new role.

Filter policies by property or policy name and press enter. 5 matches

"AmazonS3" X Clear filters

	Policy name info	Type	Description
☒	AmazonS3FullAccess	AWS m...	Provides full access to all buckets via the AWS Management Co...
☐	AmazonS3ReadOnlyAccess	AWS m...	Provides read only access to all buckets via the AWS Managem...
☐	AmazonS3OutpostsFullAccess	AWS m...	Provides full access to Amazon S3 on Outposts via the AWS Ma...
☐	AmazonS3ObjectLambdaExecutionRolePolicy	AWS m...	Provides AWS Lambda functions permissions to interact with A...
☐	AmazonS3OutpostsReadOnlyAccess	AWS m...	Provides read only access to Amazon S3 on Outposts via the A...

► **Set permissions boundary - optional** [info](#)
Set a permissions boundary to control the maximum permissions this role can have. This is not a common setting, but you can use it to delegate permission management to others.

Cancel Previous **Next**

Figure 2.17 – Add permission to the role

4. Review the role name and click **Create role**, as shown in *Figure 2.18*:

Review

Provide the required information below and review this role before you create it.

Role name*
Use alphanumeric and '*+=, @-_' characters. Maximum 64 characters.

Role description

Maximum 1000 characters. Use alphanumeric and '*+=, @-_' characters.

Trusted entities AWS service: ec2.amazonaws.com

Policies [AviController-EC2-Policy](#) [AviController-IAM-Policy](#)

* Required

Cancel Previous **Create role**

Figure 2.18 – Review and create role

5. After the IAM EC2 role is created, we need to attach it to the EC2 instance named **test**. Navigate to the EC2 dashboard and select our instance, then go to **Actions** -> **Security** -> **Modify IAM role**, as shown in *Figure 2.19*:

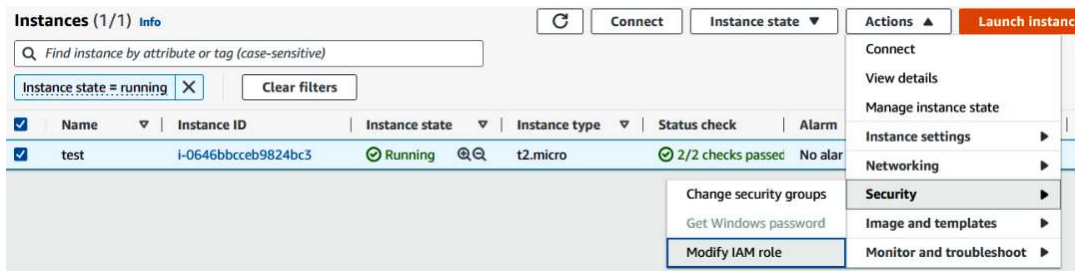


Figure 2.19 – Modify the IAM role to the instance

6. In the new window, choose the role we created earlier, and then click **Update IAM role**, as shown in *Figure 2.20*:

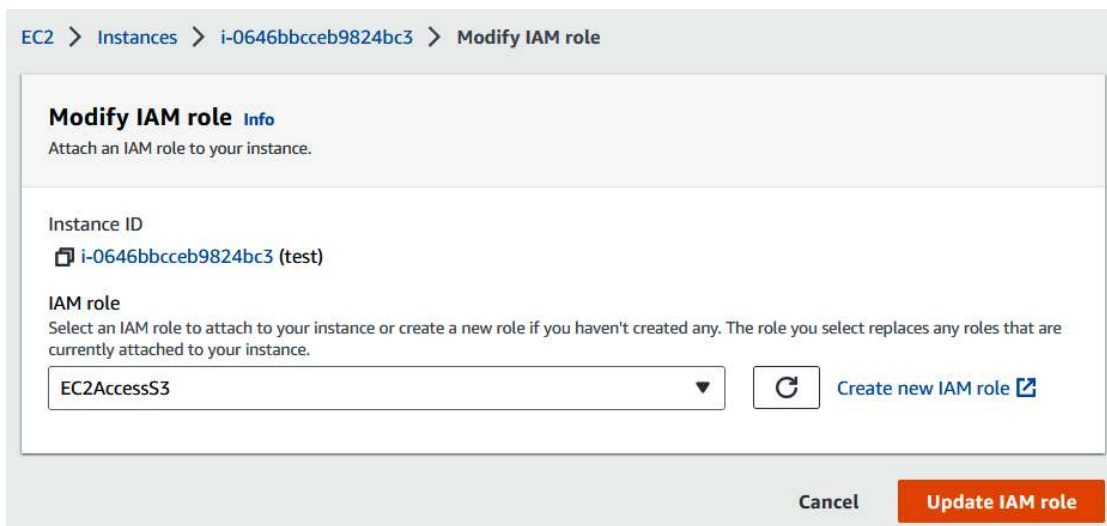


Figure 2.20 – Update the IAM role to the instance

7. Now, go back to the EC2 instance and run the following command:

```
aws s3 ls
```

Now, the S3 buckets for the AWS account will be listed, as shown in *Figure 2.21*:

```
[ec2-user@ip-172-31-28-114 ~]$ aws s3 ls
2021-04-17 20:21:43 asr-x-dll-gpt-3
```

Figure 2.21 – List the S3 bucket on the EC2 instance

By attaching the EC2 role to the EC2 instance, all the applications running on the instance will be able to assume the EC2 role's permissions. However, there is one question remaining: what if the EC2 instance does not have a public IP address? Can it still access the S3 buckets? The answer is yes, and we will explore and implement it in *Chapter 3, AWS Networking Services*.

Summary

Congratulations! We have completed *Chapter 2* of our AWS self-learn journey: *Amazon Cloud Storage Services*. In this chapter, we introduced the Amazon EBS and EFS concepts, provisioned block storage and network filesystems for EC2 instances step by step, and then discussed the Amazon S3 services and Amazon data transfer services: Snowcone, Snowball, and Snowmobile.

As you can see, AWS provides these storage solutions to meet different business needs. Amazon EBS provides block storage volumes for EC2 instances, and some use cases include running a database that needs high-performance block storage, hosting a website that requires persistent storage, or an application that requires low-latency access to data. Amazon EFS provides a shared filesystem that can be accessed from multiple EC2 instances simultaneously. Some use cases for EFS include running web applications that require shared file storage, or a big data application that requires shared storage. Amazon S3 provides object storage that is highly scalable and durable. Some use cases for S3 include storing and sharing files, images and videos, hosting a static website with static assets, and archiving data for long-term retention.

So far, we have covered the compute services and storage services from the Amazon cloud. In the next chapter, we will discuss the AWS network services. As we stated earlier, a single computer without networking is useless. It is the networks that make all the computers and services communicate and integrate to achieve business goals.

Practice questions

Questions 1-4 are based on *Figure 2.22*. You need to design an AWS storage system to move the on-prem storage to the cloud:

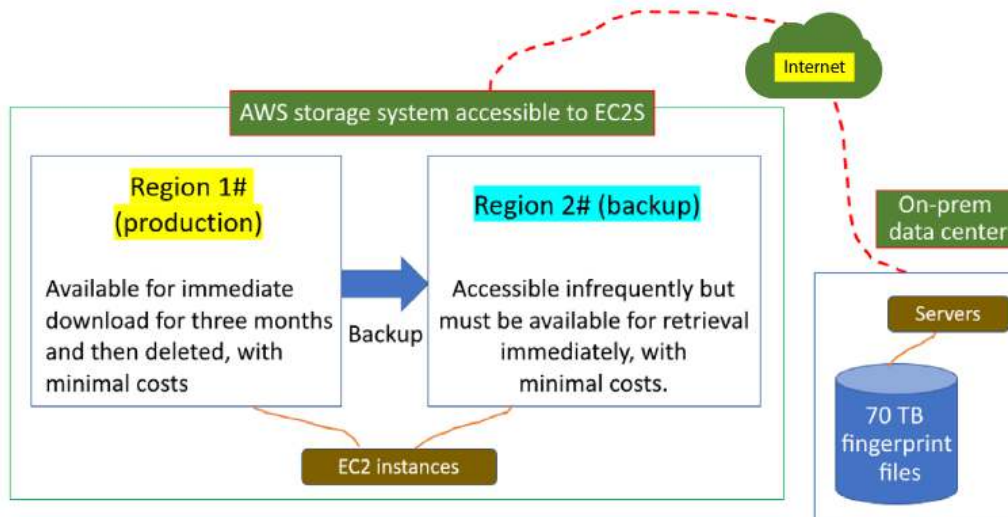


Figure 2.22 – An AWS system storage

1. How will you choose production storage in Region 1#?
 - A. EBS
 - B. S3
 - C. Glacier
 - D. EFS
2. How will you design cloud storage in Region 2#?
 - A. Amazon Glacier with expedited retrievals
 - B. Amazon S3 Standard-Infrequent Access
 - C. Amazon EFS
 - D. Amazon S3 Standard
3. What's the best way to transfer files from on-prem to Region 1#?
 - A. Amazon transfer with multi-part uploading
 - B. AWS Snowball
 - C. AWS Snowmobile
 - D. AWS Transfer using direct-connect networks

4. What's the best way to back up from Region 1# to Region 2#?
 - A. Amazon S3 Cross-Region Replication
 - B. AWS Snowball or Snowmobile
 - C. Copy the video records from the cloud to a third-party data center
 - D. Migration using direct-connect networks between regions
5. For Amazon S3, what is the largest object that can be uploaded in a single PUT?
 - A. 5 gigabytes
 - B. 5 terabytes
 - C. 500 megabytes
 - D. 5 telebytes
6. Which of the following statements is true?
 - A. EBS volumes are ephemeral.
 - B. EBS volumes can be attached to an EC2 instance in another AZ.
 - C. EBS volumes can be attached to an EC2 instance in another Region.
 - D. EBS volumes can be attached to certain EC2 instances simultaneously.
7. Company XYZ requires you to preserve, restore, and retrieve immediately every version of every file that you have stored in the AWS cloud. Which service should you use?
 - A. S3 One Zone-IA
 - B. S3 Standard with versioning enabled
 - C. S3 Glacier
 - D. S3 Reduced Storage with versioning enabled
8. An AWS engineer created an EC2 instance with a public IP address and an S3 bucket in the same Region. What do they need to do to have the instance read from and write into the S3 bucket?
9. An AWS engineer created an EC2 instance with no public IP address and an S3 bucket in the same Region. What do they need to do to enable the instance to read from and write into the S3 bucket?
10. After mounting an Amazon EFS to the EC2 instance, do you need to format mkfs to it?

Answers to the practice questions

1. B.
2. A.
3. B.
4. A.
5. A.
6. D.
7. B.
8. Create an EC2 role that has read and write access to the S3 bucket, and assign the role to the EC2 instance.
9. Create a VPC endpoint for S3. Create an EC2 role that has read and write access to the S3 bucket, and assign the role to the EC2 instance.
10. No.

Further reading

For further insights into what you've learned in this chapter, refer to the following links:

- <https://aws.amazon.com/ebs/>
- <https://aws.amazon.com/efs/>
- <https://aws.amazon.com/s3/>
- <https://docs.aws.amazon.com/ebs/index.html>
- <https://docs.aws.amazon.com/efs/index.html>
- <https://docs.aws.amazon.com/s3/index.html>